

1. The Exact TP4056 Module You Must Use

There are two versions of the TP4056 charging module sold in local electronics shops. You must buy the 6-Pad Version (with DW01A Protection IC).

- **The WRONG Module (4 Pads):** Only has IN+, IN-, BAT+, BAT-. If you use this, the ESP32 will slowly drain your 18650 battery down to 0V, permanently destroying the lithium chemistry.
- **The CORRECT Module (6 Pads):** Has IN+, IN-, B+, B-, OUT+, and OUT-. It contains two extra black chips (the DW01A protection IC and an 8205A dual-MOSFET). This module physically cuts off power to the OUT pads when the battery drops below 2.5V, saving your battery from death. **Always buy the Type-C USB version.**

2. The Updated Cheat Sheet (LDO Bypass Version)

Here is your final, corrected cheat sheet. I have completely rewritten the BOM and the Circuit Diagram from your provided PDF to incorporate the MCP1700-3302E Ultra-Low Power LDO and the 220µF Wi-Fi buffer capacitor to guarantee the 2-year battery life.

SMART MAGNETIC DOOR SENSOR & ALARM INTEGRATION
ENGINEERING REFERENCE, HARDWARE MANUAL & PRODUCTION CODE

1. System Architecture & Core Mechanism

This ultra-low power module runs for 1 to 2 years on a single 18650 lithium-ion battery. It spends 99.99% of its life in Deep Sleep (consuming only 5µA).

- **Trigger Event:** Opening or closing the door wakes the ESP32-C3 via hardware interrupt.
- **Action 1 (Instant):** Fires an ESP-NOW packet to the Master Node in <100ms.
- **Action 2 (Logging):** Connects to Wi-Fi via Static IP (saving 3 seconds of DHCP battery drain), logs the Door State and Battery Percentage to Home Assistant via MQTT, and immediately returns to Deep Sleep.
- **Security Matrix:** If the Home Assistant "Lock Home" toggle is armed, the Master Node intercepts the ESP-NOW packet and instantly triggers the 120dB industrial siren and Raspberry Pi SIM call/SMS system.

2. Bill of Materials (BOM)

Component	Engineering Function	Technical Notes
ESP32-C3 SuperMini	Core Microcontroller	Uses RISC-V ultra-low power deep sleep state.
MC-38 Magnetic Reed	Door Open/Close Detector	Zero-power mechanical

Switch		switch.
18650 Li-Ion Battery	Power Source	Salvaged from old power bank ($2000\text{mAh} + \$$ recommended).
TP4056 (6-Pad Version)	Charger & Protection	Critical: Must have OUT+/OUT- pads. Prevents battery explosion and over-discharge ($<2.5\text{V}$).
MCP1700-3302E	Ultra-Low Power LDO	Steps 4.2V down to 3.3V cleanly. Quiescent current is only $1.6\mu\text{A}$ (Replaces Boost Converter).
$220\mu\text{F}$ Electrolytic Cap	Wi-Fi Spike Buffer	Supplies instant 350mA burst for Wi-Fi ping so the LDO doesn't crash the MCU.
$10\text{k}\Omega$ Resistor	Wake-Up Pull-Up	Pulls GPIO 2 HIGH the instant the door opens.
2x $1\text{M}\Omega$ Resistors	Battery Voltage Divider	Cuts 4.2V battery to 2.1V safely for ESP32 ADC. Drains only $2\mu\text{A}$.
$0.1\mu\text{F}$ (104) Ceramic Cap	ADC Noise Filter	Smooths out Wi-Fi transmission noise for accurate battery %.
ABS Plastic Project Box	IP65 Enclosure	Size $60\text{x}36\text{x}25\text{mm}$. Houses the battery and MCU neatly.

3. Complete Circuit & Wiring Schematic

 **SAFETY WARNING:** Never connect an 18650 battery directly to the ESP32. Always route

power through the TP4056 B+/B- and OUT+/OUT- pads to ensure under-voltage protection!
Do not use the 5V / VIN pin on the ESP32.

Plaintext

ULTRA-LOW POWER DOOR SENSOR + BATTERY TELEMETRY WIRING (LDO BYPASS)





4. Updates Required on MASTER NODE (master-node.ino)

You must add these snippets to your existing Master Node code so it can intercept the door packet and trigger the siren if the house is locked.

1. Update the Struct (Near top of code):

```
C++
// Replace the old struct_message with this to accept Battery Data
typedef struct struct_message {
    char modelID[20]
    bool lightActive
    uint8_t nodeID
    uint8_t batteryPct // <--- NEW ADDITION!
} struct_message

bool isSystemArmed = false // Add this global variable for HA Security State
```

2. Update mqttCallback() (To listen to Home Assistant Lock):

```
C++
// Add inside mqttCallback()
if strcmp(topic, "home/alarm/system_state") == 0
    isSystemArmed = (cmd == "ARMED")
    Serial printf "[SECURITY] Alarm System is now: %s\n" isSystemArmed ? "ARMED"
    "DISARMED"
```

3. Update reconnectMQTT() (To subscribe to Lock State):

```
C++  
// Add inside reconnectMQTT() after successful connection  
mqttClient.subscribe("home/alarm/system_state")
```

4. Update onESPNowDataRecv() (To trigger the Siren):

```
C++  
// Add inside onESPNowDataRecv()  
if (strcmp(incomingSlaveData.nodeID, "MAIN_DOOR") == 0)  
    bool doorIsOpen = incomingSlaveData.lightActive  
    if (doorIsOpen)  
        Serial.println("[SECURITY] FRONT DOOR OPENED!")  
        if (isSystemArmed())  
            triggerEmergencySiren("Intruder at Front Door!")  
    }  
}
```

5. Production Firmware for DOOR NODE (door-node.ino)

Upload this code to the ESP32-C3 Door Sensor. (Prerequisite: Arduino ESP32 Core v2.0.3+ for esp_sleep functions).

```
C++  
#include <WiFi.h>  
#include <PubSubClient.h>  
#include <esp_now.h>  
#include <esp_sleep.h>
```

```

// =====
// 1. CONFIGURATION
// =====
#define DOOR_SENSOR_PIN 2 // RTC GPIO 2 for Wakeup
#define BATTERY_ADC_PIN 1 // GPIO 1 for Battery Voltage Divider
#define DOOR_NODE_ID "MAIN_DOOR"

const char WIFI_SSID = "Syndicate" |cite: 6
const char WIFI_PASSWORD = "586792023-" |cite: 6
const char MQTT_SERVER = "192.168.0.40" |cite: 6
const int MQTT_PORT = 1883 |cite: 6
const char MQTT_USER = "redwanmqtt" |cite: 6
const char MQTT_PASS = "abcd2005-" |cite: 6

// STATIC IP SAVES ~3 SECONDS OF WI-FI DHCP DRAIN
IPAddress local_IP(192, 168, 0, 50) |cite: 6
IPAddress gateway(192, 168, 0, 1) |cite: 6
IPAddress subnet(255, 255, 255, 0) |cite: 6

uint8_t masterMacAddress[] = { 0x68, 0xFE, 0x71, 0x8B, 0x58, 0x50 } |cite: 6

WiFiClient espClient |cite: 6
PubSubClient mqttClient(espClient) |cite: 6

typedef struct struct_message { |cite: 6
    char model[20] |cite: 6
    bool lightActive |cite: 6
    uint8_t node |cite: 6
    uint8_t batteryPct |cite: 6
    struct_message |cite: 6
    struct_message doorTelemetry |cite: 6

uint8_t calculateBatteryPercentage() |cite: 6
uint32_t adc_mv = analogRead(MMIVOLTS(BATTERY_ADC_PIN)) |cite: 6
float battery_voltage = (adc_mv / 2.0 / 1000.0 // x2 for 1M/1M divider
Serial.printf "[BATTERY] Raw Voltage: %.2fV\n", battery_voltage) |cite: 6

// Map 18650 Li-Ion discharge curve (4.2V = 100%, 3.3V = 0%)
int pct = map(battery_voltage, 100, 330, 420, 0, 100) |cite: 6
pct = constrain(pct, 0, 100) |cite: 6
return uint8_t(pct) |cite: 6
|cite: 6

```

```

void setup() [cite: 6]
{
    setCpuFrequencyMhz(80) // Save power during brief awake time
    Serial.begin(115200) [cite: 6]

    pinMode(DOOR_SENSOR_PIN, INPUT); // Relies on external 10k pull-up
    analogReadResolution(12); // 12-bit ADC

    // 1. Read States[cite: 6]
    bool isDoorOpen = (digitalRead(DOOR_SENSOR_PIN) == HIGH); [cite: 6]
    uint8_t currentBatPct = calculateBatteryPercentage(); [cite: 6]

    Serial.printf("\n[WAKEUP] Door: %s | Battery: %d%%\n", isDoorOpen ? "OPEN" : "CLOSED",
currentBatPct); [cite: 6]

    // 2. Initialize ESP-NOW[cite: 6]
    WiFi.mode(WIFI_STA); [cite: 6]
    if (esp_now_init() == ESP_OK) [cite: 6]
    {
        esp_now_peer_info_t peerInfo = {}; [cite: 6]
        memcpy(peerInfo.peer_addr, masterMacAddress, 6); [cite: 6]
        peerInfo.channel = 0; [cite: 6]
        peerInfo.encrypt = false; [cite: 6]
        esp_now_add_peer(&peerInfo); [cite: 6]

        strcpy(doorTelemetry.nodeID, DOOR_NODE_ID); [cite: 6]
        doorTelemetry.lightActive = isDoorOpen; [cite: 6]
        doorTelemetry.mode = 0; [cite: 6]
        doorTelemetry.batteryPct = currentBatPct; [cite: 6]

        // FIRE INSTANTLY TO MASTER NODE[cite: 6]
        esp_now_send(masterMacAddress, (uint8_t *)&doorTelemetry,
sizeof(doorTelemetry)); [cite: 6]
        Serial.println("[ESP-NOW] Packet fired to Master Node."); [cite: 6]
    } [cite: 6]

    // 3. Fast Wi-Fi Connection[cite: 6]
    WiFi.config(local_IP, gateway, subnet); [cite: 6]
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD); [cite: 6]

    unsigned long startAttempt = millis(); [cite: 6]
    while ((WiFi.status() != WL_CONNECTED) && millis() - startAttempt < 4000) { [cite: 6]
        delay(50); [cite: 6]
    } [cite: 6]

```

```

// 4. MQTT Logging with JSON (Includes Battery Data)[cite: 6]
if (WiFi.status() == WL_CONNECTED) {cite: 6
  mqttClient.setServer(MQTT_SERVER, MQTT_PORT);cite: 6
  if (mqttClient.connect(DOOR_NODE_ID, MQTT_USER, MQTT_PASS)) {cite: 6
    mqttClient.publish("home/door/main/state" isDoorOpen ? "ON" "OFF" true cite: 6

    char batStr 8 cite: 6
    sprintf(batStr, sizeof(batStr), "%d", currentBatPct);cite: 6
    mqttClient.publish("home/door/main/battery" batStr true cite: 6

    Serial.println("[MQTT] State & Battery logged to Home Assistant." cite: 6
  }cite: 6
}cite: 6

// 5. CONFIGURE NEXT WAKE-UP & GO TO DEEP SLEEP[cite: 6]
if (isDoorOpen) {cite: 6
  esp_deep_sleep_enable_gpio_wakeup(1ULL << DOOR_SENSOR_PIN,
ESP_GPIO_WAKEUP_GPIO_LOW);cite: 6
}
else cite: 6
  esp_deep_sleep_enable_gpio_wakeup(1ULL << DOOR_SENSOR_PIN,
ESP_GPIO_WAKEUP_GPIO_HIGH);cite: 6
}cite: 6

// Kill internal LED before sleeping[cite: 6]
pinMode(8, OUTPUT);digitalWrite(8, HIGH);cite: 6

Serial.println("[SLEEP] Entering Deep Sleep ($5uA)..." cite: 6
delay(10) cite: 6
esp_deep_sleep_start();cite: 6
}cite: 6

void loop() // Never runs[cite: 6]

```

6. Home Assistant Configuration (configuration.yaml)

Add this configuration to integrate the Door Sensor, Battery Telemetry, and the Master "Lock Home" Security Automations[cite: 6].

YAML

```

mqtt:
  binary_sensor:
    - name: "Main Door Status"
      state_topic: "home/door/main/state"
      payload_on: "ON"
      payload_off: "OFF"
      device_class: door

  sensor:
    - name: "Main Door Battery"
      state_topic: "home/door/main/battery"
      unit_of_measurement: "%"
      device_class: battery
      state_class: measurement

  input_boolean:
    lock_home_alarm:
      name: "Lock Home (Arm Alarm)"
      icon: mdi:shield-lock

  shell_command:
    # Triggers your Raspberry Pi Python Script for GSM Call/SMS
    trigger_sim_sms: 'ssh pi@192.168.0.XX "python3 /home/pi/emergency_sim_alert.py"'

```

Home Assistant Automations (automations.yaml)

1. Sync Armed State to Master Node:

[cite: 6]

```

YAML
- alias: "Security: Sync Alarm State to Master Node"
  trigger:
    - platform: state
      entity_id: input_boolean.lock_home_alarm
  action:
    - service: mqtt.publish
  data:
    topic: "home/alarm/system_state"
    payload: "{{ 'ARMED' if is_state('input_boolean.lock_home_alarm', 'on') else 'DISARMED' }}"
  retain: true

```

2. Intruder Alert (App Push + RPi GSM SMS/Call):

[cite: 6]

YAML

```
- alias: "Security: INTRUDER DOOR BREACH"
  trigger:
    - platform: state
      entity_id: binary_sensor.main_door_status
      to: "on"
  condition:
    - condition: state
      entity_id: input_boolean.lock_home_alarm
      state: "on"
  action:
    - service: notify.notify
  data:
    title: "🔴 CRITICAL SECURITY ALERT 🔴"
    message: "The Main Door has been opened while the system is Armed!"
    - service: shell_command.trigger_sim_sms
```

7. Fabrication & Mounting Instructions

1. **The Enclosure:** Purchase a small white ABS Project Box (36x25mm). It is perfectly sized to fit the 18650 battery cell, the TP4056 board, and the ESP32-C3 tightly[cite: 6].
2. **USB-C Charging Hole:** Cut a tiny, precise rectangular slit in the side of the ABS box exposing only the USB-C port of the TP4056 module[cite: 6]. Once every 1–2 years, simply plug a standard USB-C phone charger or power bank directly into the door module for 3 hours to recharge it from 0% to 100%[cite: 6].
3. **Reed Switch Exiting:** Drill a tiny 3mm hole on the side of the box to thread the two wires of the MC-38 Reed Switch outward[cite: 6].
4. **Mounting:** Use thick 3M VHB double-sided tape to attach the ABS box high on the stationary door frame[cite: 6]. Stick the small magnetic block on the swinging door, ensuring it rests within 5mm of the wired reed switch when the door is fully shut[cite: 6].